Agent vs. harness

One describes goal-directed behavior; the other provides the machinery.

Three layers, three responsibilities

MODEL

The LLM generates responses and proposes actions. It does not itself execute a filesystem operation or API request.

AGENT

A goal-directed system that uses a model and tools to act, observe results, and decide what to do next.

HARNESS

The software that calls the model, executes permitted tools, returns results, manages state, and enforces limits.

A concrete example: fix a login bug

Agent behavior: inspect the code, propose a fix, edit, run tests, and revise based on the results.

Harness responsibilities: expose file and command tools, check permissions, execute calls, preserve conversation state, and stop when a budget or time limit is reached.

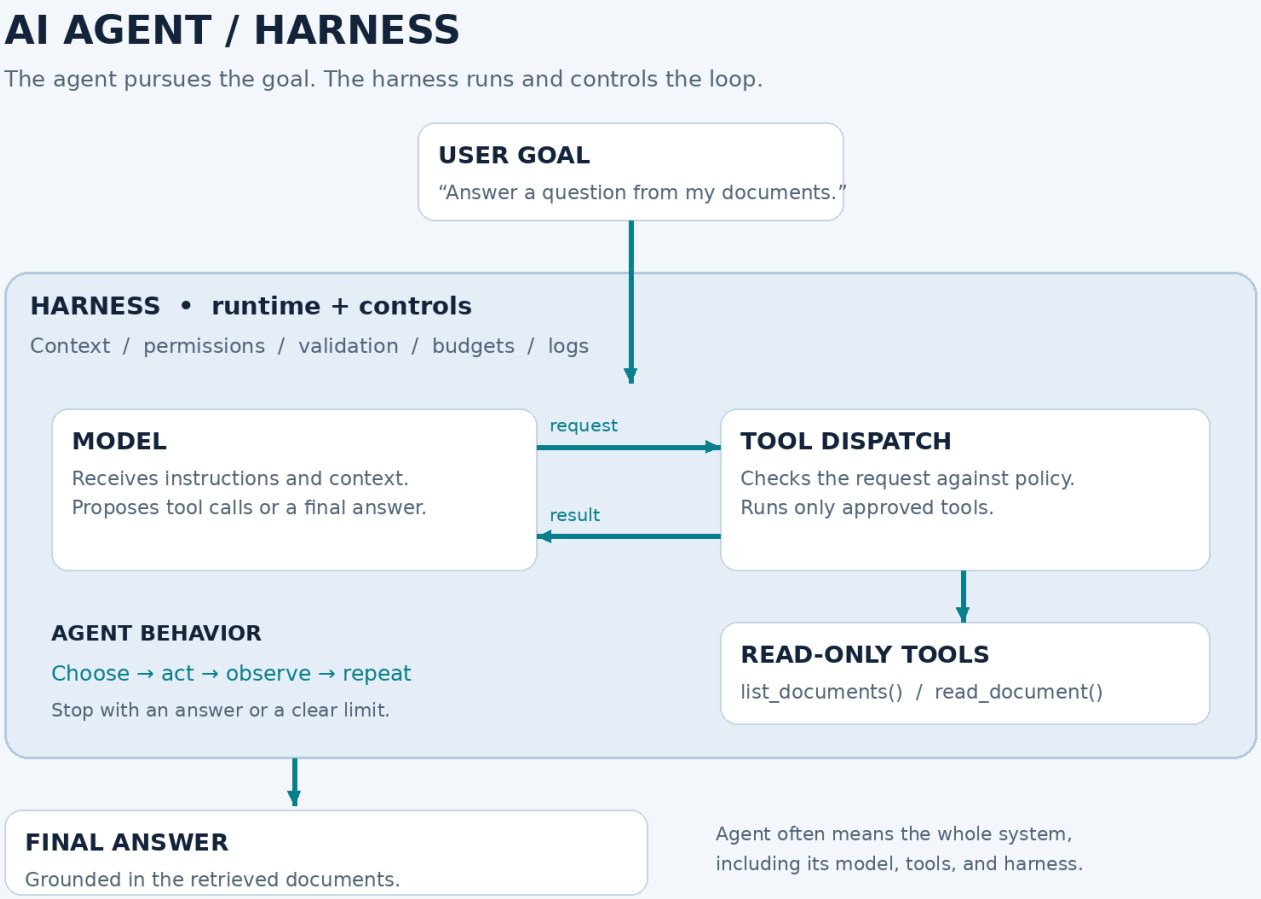
The key distinction

The model requests an action. The harness decides whether it may run and executes it. Safety boundaries must exist in code, not only in the prompt.

Terminology varies: people often call the complete package an “agent.” The harness is part of that package, not a competing alternative.

How the pieces connect

A small document assistant needs one model, two tools, and a controlled loop.



Follow one request

1. A user asks a question; the harness supplies instructions and tool schemas.

2. The model requests a document; the harness validates and runs the tool.

3. The tool result returns to the model with the matching tool-call identifier.

4. The model continues or answers; the harness stops at a defined limit.

Document contents are untrusted data. They cannot authorize new tools, change permissions, or override the user’s task.

Start small. Add control.

Example goal: answer questions using documents in one approved folder.

Minimum ingredients

Choose a tool-capable model API. Define list_documents() and read_document(name) with strict argument schemas. Add instructions to answer from the documents and admit when evidence is missing.

The harness loop (provider-neutral pseudocode)

```
history = [instructions, question]
for step in range(10):
    reply = model.generate(history, tool_schemas)
    history.append(reply)
    if not reply.tool_calls:
        return reply.text
    for call in reply.tool_calls:
        args = validate_schema(call)
        check_allowlist_paths_and_permissions(call, args)
        result = execute_with_timeout(call.name, args)
        history.append(tool_result(call.id, result))
return "Stopped: step limit reached."
```

This is a sketch, not runnable SDK code. Your API adapter must preserve tool-call IDs, handle errors and refusals, and respect token and spending budgets.

Before expanding its capabilities

- Constrain file paths to the approved folder, including symlink resolution.
- Enforce tool allowlists, argument validation, deadlines, and cost limits.
- Require explicit approval for destructive or externally visible actions.
- Record actions without secrets; test answerable and unanswerable questions.

Start with one agent and read-only tools. Add memory, write access, or multiple agents only when the task actually needs them.